

האוניברסיטה העברית

בי"ס להנדסה ולמדעי המחשב

בחינה בקורסי מבוא למדעי המחשב:

67101 – מבוא למדעי המחשב

67130 – מבוא למדעי המחשב בדגש על תכנות פרוצדוראלי

תאריך: 31.7.2005
משך הבחינה: שלוש שעות

מועד ב' - סמסטר א' - תשס"ה – 2004-2005
המורה: פרופ' ג'ף רוזנשיין

מילוי פרטים אישיים:

מספר מחברת: _____

מספר ת"ז (9 ספרות): _____

הנחיות:

1. בתום הבחינה יש להגיש שאלון זה יחד עם מחברת הבחינה.
2. יש לענות על 3 מתוך 4 השאלות הבאות (3 בלבד). משקל כל השאלות זהה (33 נק').
3. הקפידו הקפדה יתרה על סדר וכתב יד קריא. בחינה בלתי קריאה לא תזכה את כותבה במלוא הנקודות.
4. יש להתחיל את התשובה לכל שאלה בעמוד חדש, ולציין בראש העמוד בצורה ברורה את מספר השאלה.
5. יש למחוק טיוטות באופן ברור.
6. יש להשאיר שוליים.
7. אסור להשתמש בעט אדום.
8. הקפידו על סגנון תכנותי נאות: כתבו באופן מודולארי, אל תכפילו קוד וכו'.
9. תעדו כל תכנית כך שניתן יהיה להבין בנקל (מותר לתעד בעברית).
10. פתרון מיטבי לשאלה יחשב פתרון פשוט ויעיל ככל האפשר. תוכנית מסורבלת או מסובכת לא תתקבל כתשובה מלאה, גם אם היא נכונה!
11. אין להשתמש במבני נתונים פרט לאלו שהוגדרו בכל שאלה. עם זאת, מותר להשתמש במבני נתונים נוספים אשר גודלם או מספרם אינו תלוי בקלט (למשל מותר להקצות מערך בגודל 1).

בחירת השאלות לבדיקה:

יש לסמן V ליד השאלות שברצונך שייבדקו: לכל היותר 3 סימני V; לא תיבדקנה שאלות שלא מסומן לידן V.

מספר שאלה	סמן V ליד שאלה שברצונך שתיבדק	לשימוש הבודק
1		
2		
3		
4		

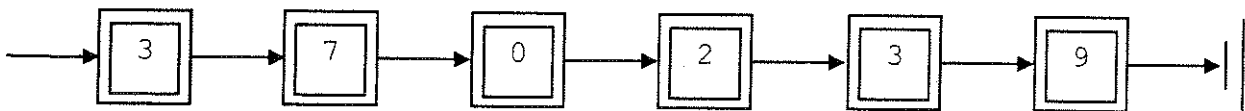
שאלה 1

בתכנית מוגדרת רשימה מקושרת חד כיוונית באמצעות המבנה:

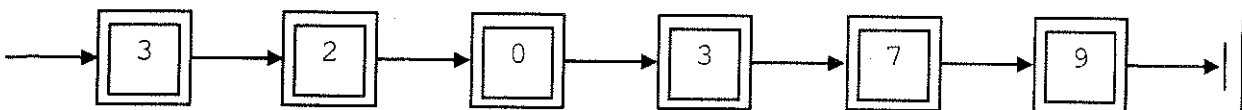
```
class Node {  
    int data;  
    Node next;  
}
```

א. כתוב פונקציה סטטית בשם `partition` (השייכת למחלקה ללא `data-members`) אשר מקבלת מצביע (מטיפוס `Node`) לראשה של רשימה לא ריקה כנ"ל, ומסדרת את הרשימה שהתקבלה כך שכל האיברים ברשימה שקטנים או שווים לאיבר הראשון, יהיו משמאלו, וכל האיברים הגדולים מהאיבר הראשון, יהיו מימינו. הפונקציה תחזיר מצביע לראש רשימה מסודרת כנדרש. (23 נק')

לדוגמא, בהינתן הרשימה הבאה:



הפונקציה `partition` תסדר אותה באופן הבא:



כך ש- "3" שהיה האיבר הראשון (השמאלי ביותר) ברשימה העליונה, הפך להיות האיבר הרביעי ברשימה התחתונה: משמאלו ממוקמים האיברים שקטנים או שווים לו (3,2,0), ומימינו ממוקמים האיברים הגדולים ממנו (7,9).

הנחיות:

- כרגיל, בסעיף זה אסור להקצות זיכרון (כלומר אסור להשתמש ב- `new`, ראה סעיף 11 בהנחיות שבתחילת הבחינה).
- אין חובה לשמור על סדר האיברים היחסי כפי שהיה ברשימה המקורית, כל עוד הרשימה המסודרת מקיימת את התנאים שפורטו לעיל.

ב. מהו סדר גודל זמן הריצה של הפונקציה שכתבת? (המקרה הגרוע). (הסבר). (5 נק')

ג. תחת ההנחיות וההנחות שבסעיף א', כתוב פונקציה דומה בשם `partition2`, אשר בנוסף לרשימה, מקבלת גם ערך `k` – של קודקוד כלשהו מתוך הרשימה (מטיפוס `int`), ומסדרת את הרשימה כך שכל האיברים שקטנים או שווים ל- `k` ימוקמו משמאלו וכל האיברים שגדולים ממנו ימוקמו מימינו. (5 נק')

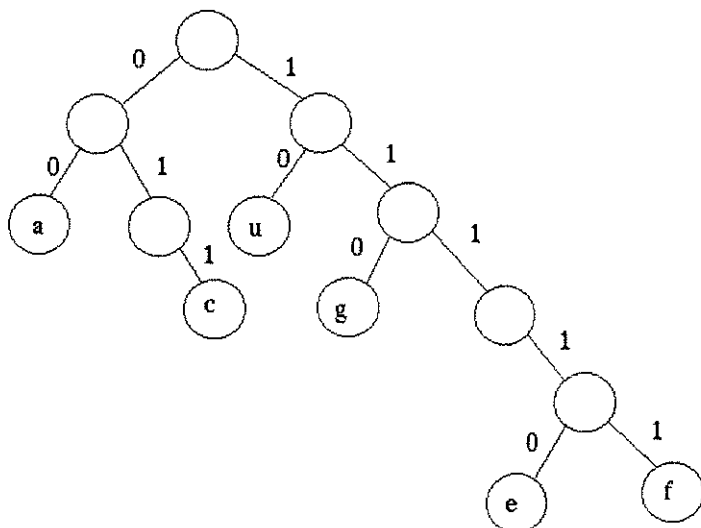
הנחיות:

- ניתן להניח כי קיים לפחות מופע אחד של `k` ברשימה.
- אם קיים יותר ממופע אחד, אין זה משנה איזה מבין מופעי ה- `k` בוחרים.

שאלה 2

- "עץ פענוח" הוא עץ חיפוש בינארי בו כל מסלול (מן השורש אל עלה כלשהו) מייצג "מילת קוד בינארית" (שמכילה רק אפסים ואחדים). מילת הקוד משויכת לתו שנמצא בעלה שאליו הוביל המסלול. מילת קוד נכתבת משמאל לימין, כאשר כל פנייה שמאלה מוסיפה "0", וכל פנייה ימינה מוסיפה "1".

לדוגמא, להלן עץ פענוח (שבעליו התווים a/c/u/g/e/f) ומילות הקוד שמייצגות כל תו:



a: 00
c: 011
u: 10
g: 110
e: 11110
f: 11111

בהינתן מחרוזת בינארית כלשהי, ניתן לנסות "לפענח" אותה באמצעות עץ הפענוח. למשל, ע"פ העץ משמאל, פענוח המחרוזת "01111111110" הוא "cfg" שכן:

$\overbrace{011}^c \overbrace{1111}^f \overbrace{1110}^g$

כמובן שיש מחרוזות שלא ניתן לפענח באמצעות העץ הנ"ל, למשל: "01001" או "11011" או "1110".

- בהינתן מחרוזת בינארית s, אזי "מחרוזת מצומצמת" של s היא מחרוזת שהתקבלה ע"י מחיקת תו בודד של s. לדוגמא, להלן כל המחרוזות המצומצמות של המחרוזת s="01101" (x מסמן את התו שנמחק):

"0110" <= 0110x .I
"0111" <= 011x1 .II
"0101" <= 01x01 .III
"0101" <= 0x101 (הופיע במילה הקודמת) .IV
"1101" <= x1101 .V

- בתוכניתנו, עץ פענוח מוגדר באמצעות המבנה:

```
class Node {
    char    data;
    Node    left;
    Node    right;
}
```

כאשר קודקודים פנימיים מכילים את התו רווח (' '), שמוגדר כלא שייך למחרוזות השונות.

א. כתוב פונקציה סטטית בשם decode (ששייכת למחלקה ללא data-members) המקבלת מחרוזת בינארית (מטיפוס String) בשם s, ומצביע (מטיפוס Node) לשורש של עץ פענוח כלשהו. הפונקציה מחזירה את רצף התווים (String) הקצר ביותר האפשרי, שמהווה פענוח של מחרוזת מצומצמת כלשהי של s. אם לא קיים כזה רצף, הפונקציה תחזיר את המחרוזת הריקה ("").

לדוגמא, עבור המחרוזת $s = "01111111110"$ והעץ שבדוגמא, קיימות בדיוק 3 מחרוזות מצומצמות שונות:

(1) 0111111111 שלא ניתן לפענח.

(2) $\overbrace{11111}^f \overbrace{11110}^e$ אשר פענוחה הוא: "fe".

(3) $\overbrace{011}^c \overbrace{11111}^f \overbrace{10}^u$ אשר פענוחה הוא: "cfu".

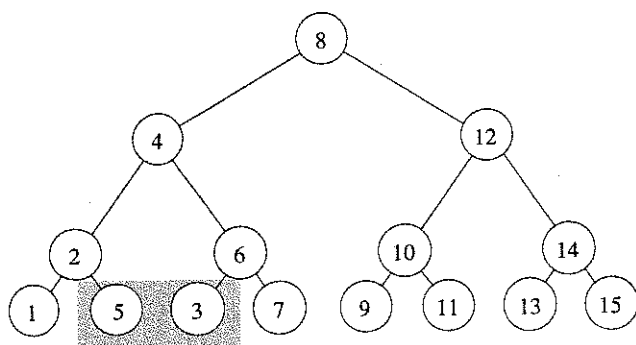
כיוון ש-"fe" קצרה מ-"cfu", התשובה שתחזיר הפונקציה decode בעבור המחרוזת s היא "fe".

הנחיות:

- מותר להניח שהקלט תקין (s מכילה רק אפסים ואחדים).
- קבלת התו ה-i של מחרוזת s מתבצעת ע"י: `s.charAt(i)`
- קבלת תת-המחרוזת של s שבין התווים i ו-j (כולל) מתבצעת ע"י: `s.substring(i, j+1)` (למשל `"smiles".substring(1, 5)` מחזיר "mile").

ב. מהו סדר גודל זמן הריצה של הפונקציה שכתבת? (המקרה הגרוע). (הסבר). (5 נק')

שאלה 3



- עץ חיפוש בינארי יקרא "עץ מוחלף" אם מחליפים זוג יחיד של קודקודים שלו. הקודקודים אינם ממוקמים באותו ענף (שמתחיל בשורש). כלומר ההחלפה אינה בין אב לבן, סב לנכד, סב-רב לנין, וכו'.

- לדוגמא, בעץ משמאל הוחלפו הקודקודים 5 ו-3.

- בתוכניתנו, עץ חיפוש בינארי מוגדר באמצעות המבנה:

```
class Node {
    int data;
    Node left;
    Node right;
}
```

א. כתוב פונקציה סטטית בשם fix (ששייכת למחלקה ללא data-members) המקבלת שורש של עץ חיפוש בינארי (טיפוס Node) תקין או מוחלף. אם העץ מוחלף, הפונקציה תתקן אותו ותחזיר true. אחרת, הפונקציה תחזיר false. (25 נק')

- מהו סדר גודל זמן הריצה של הפונקציה שכתבת במקרה הגרוע ביותר? (הסבר). (4 נק')
- מהו סדר גודל זמן הריצה של הפונקציה שכתבת במקרה הטוב ביותר? (הסבר). (4 נק')

שאלה 4

כל סעיפי שאלה זו מתייחסים לשפת התכנות Java. ענה על 8 מתוך 10 הסעיפים הבאים. משקל כל הסעיפים זהה. בכל סעיף יש להקיף תשובה אחת בלבד (אם שיניתם תשובתכם יש למחוק את התשובה הקודמת באופן ברור שאינו משתמע לשני פנים).

מונחים בהם נעשה שימוש בשאלה זו:

מינח:	מצביע	מחלקה	ממשק	חבילה	העמסה	החלפה	בנאי	המרה	פונקציה
פירוש:	reference	class	interface	package	overload	Override	constructor	cast	Method

א. בהתייחס לקטע הקוד הבא:

```
class E extends Exception { /*...*/ }
void f() throws E { /*...*/ } // defined in some class
```

נניח שפונקציה g קוראת לפונקציה f הנ"ל. איזו מבין הטענות הבאות נכונה?

- יש להצהיר על g כפונקציה שעלולה לזרוק את E (או מחלקה שיורשת מ E).
- במימוש של g חייבת להופיע המילה השמורה try.
- בהכרח (i) או (ii).
- בהכרח (i) ו-(ii).
- אין תשובה נכונה.

ב. בהינתן קטע הקוד הבא:

```
class A { A(int i) { System.out.println(i); } }
class B extends A { B(int i) { A a=new A(2); System.out.println(i); } }
```

מה תדפיס הפקודה (הפלט נקרא משמאל לימין):

- 5 2
- 5
- 5 2 5
- 2 2 5
- 2 5
- אין תשובה נכונה.

ג. נתון כי פונקציה f משתמשת במילה השמורה try ואיננה משתמשת במילה השמורה finally. האם ב-f מופיעה המילה השמורה catch?

- לא.
- נתון לשיקול דעתו של המשתמש.
- כן.
- תלוי במשימה שעל f לבצע.

ד. איזו מסקנה ניתן להסיק מקטע הקוד הבא:

```
static void f(X a, X b) { X tmp=a; a=b; b=tmp; }
```

- המצביעים a ו-b שונים מ-null.
- f מוגדרת בתוך המחלקה X.
- אחרי ש-f הסתיימה (חזרנו לפונקציה שקראה לה), תוכן שני המצביעים שנשלחו ל-f הוחלף.
- f הייתה צריכה להיות מוגדרת כפונקציה שאיננה סטטית.
- אף תשובה לא נכונה.

ה. איזו מהטענות הבאות נכונה:

- אם קיימת פונקציה אבסטרקטית במחלקה, אז המחלקה כולה אבסטרקטית.
- אם מחלקה היא אבסטרקטית (abstract class) אז יש לה לפחות פונקציה אבסטרקטית אחת.
- אם כל הפונקציות במחלקה אבסטרקטיות, אז ניתן לייצג את המחלקה כממשק.
- כל התשובות הנ"ל נכונות.
- אין תשובה נכונה.

- ו. תהי Y מחלקה שמממשת שני ממשקים שונים: I1 ו-I2. איזו מבין הטענות הבאות נכונה?
- (i) אי אפשר להגדיר מצביע ל-I1 או ל-I2 שכן ממשקים הם אבסטרקטים.
 - (ii) לעיתים מותר להמיר מצביע מטיפוס I1 ל-I2 באמצעות המרה לא מפורשת.
 - (iii) תמיד ניתן יהיה להמיר (cast) בהצלחה מצביע מטיפוס I1 ל-I2, שכן Y מממשת את שניהם.
 - (iv) אפשר להקצות אובייקט מסוג I1 או I2 (לדוגמא: new I1()) שכן I1 ו-I2 כבר מומשו ע"י Y.
 - (v) אין תשובה נכונה.

- ז. נניח כי a ו-b הינם int-ים חיוביים. למה שווה הביטוי: $a - (a/b) * b$?
- (i) a - ל
 - (ii) a % b - ל
 - (iii) a * b - ל
 - (iv) 0 - ל
 - (v) אין תשובה נכונה.

- ח. איזו מבין הטענות הבאות אינה נכונה ביחס ל- System.out (בהנחה שמעולם לא ראית את ההגדרה האמיתית; זיכרו כי המוסכמה ששם של מחלקה יתחיל באות גדולה, היא רק מוסכמה):
- (i) יתכן שזוהי מחלקה שמקננת בתוך החבילה System.
 - (ii) יתכן שזוהי data-member של החבילה System.
 - (iii) יתכן שזוהי data-member של המחלקה System.
 - (iv) יתכן שזוהי מחלקה שמקננת (inner class) בתוך המחלקה System.
 - (v) בחר/י בסעיף זה אם כל הטענות הנ"ל נכונות.

ט. מה יהיה הפלט של g()?

```
class B { void f() { System.out.print( "B" ); } }
class D extends B { void f() { System.out.print( "D" ); }
                    void g() { B b = this;
                               System.out.print( b instanceof B );
                               System.out.print( b == this );
                               b.f();
                               this.f();
                               super.f(); } }
```

- (i) true true DDD
- (ii) false true BDB
- (iii) true true DDB
- (iv) true true BDB
- (v) false true DDB
- (vi) false true DDD
- (vii) אין תשובה נכונה.

י. בהתייחס לקטע הקוד הבא:

```
final int f() { /* implementation */ }
```

- איזו מבין הטענות הבאות נכונה?
- (i) לא ניתן להחליף את f (לא ניתן לבצע override).
 - (ii) אפשר להוסיף את המילה static מייד אחרי ה- final (ולפני ה- int).
 - (iii) הערך ש- f מחזירה הוא קבוע.
 - (iv) לא ניתן להעמיס על f (לא ניתן לבצע overload).
 - (v) כל התשובות הנ"ל נכונות.
 - (vi) אין תשובה נכונה.

בהצלחה!